

Lab 4

Joohe Oh

02-08-2023

Submission Instructions

This lab (and all labs) will be graded for correctness. To submit the lab:

1. Once all code is completed and saved, in the RStudio ribbon, click Knit > Knit to PDF.
2. If you have trouble knitting **anything** to PDF, please work with the Instructor or GSI to resolve the issue. It is okay to instead knit to HTML, then convert to PDF using Adobe or other software.
3. If there is any handwritten work for the lab, combine the handwritten work with the knitted PDF into a single PDF file.
4. Submit the PDF to Gradescope, **selecting** pages for questions. *You will lose half a point on the lab if you don't select pages.*

Lab 4 is due to Gradescope by Sunday, February 12th at 11:59PM PT.

Today's lab will leverage simulated data to demonstrate the pitfalls of (unadjusted) R-squared as a model selection tool, and the troubles that highly correlated covariates can cause for interpretation and model variance.

In particular, you will learn how to: * script the simulation pipeline; * consider validation prediction performance as an alternative model evaluation tool.

Part I: Adjusted R-squared

Recall that R-squared (ie the proportion of variance explained by the regression line) can only improve as we include additional covariates into the model. Ideally, we can modify the R-squared metric (R_{adj}^2) to penalize additional covariates that do not provide additional modeling benefit (for example, if we include a random noise covariate, ϵ , into the model, this should leave R_{adj}^2 unchanged in expectation. The particular expression for such a quantity is as follows:

$$R_{adj}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - (p + 1)}$$

Q1: Prove that (R_{adj}^2) is upper bounded by R^2 for a given model.

$$\frac{n-1}{n-(p+1)} \geq 1$$

$$R_{adj}^2 \leq 1 - (1 - R^2) = R^2$$

Now, let's develop a simulation that will help us study the impact of using adjusted R-squared. First, we will simulate our usual data generating model for linear regression:

$$\vec{y} = X\vec{\beta} + \vec{\epsilon}, \quad \vec{\epsilon} \sim \mathcal{N}(0, \sigma^2 \mathbb{I})$$

Please feel free to spend some time learning what some of the above LaTeX commands accomplish for us. For example:

- what does `\sim` do?
- what does `\quad` do?
- what do `\mathcal{\{}}` and `\mathbb{\{}}` do?
- In the above bullet, I include an extra backslash, because backslash is the **escape** character for R Markdown.

Thanks for bearing with this typesetting aside. Now, let us create a simulation function. While for modeling purposes, we often treat X as fixed, we will randomly generate this data for simulation as random standard normal entries.

Q2: complete the function `generateX()`, which will simply return an $n \times (p + 1)$ matrix, where the first column is the *1-vector*, and the other columns are standard normal entries. Note that the object names I write may help guide you.

```
generateX <- function(n,p) {
  data_vec <- rnorm(n*p)
  data_matrix_no_intercept <- matrix(data_vec, nrow = n)
  data_matrix <- cbind(rep(1,n), data_matrix_no_intercept)
  return(data_matrix)
}

generateX(50,6)
```

```
##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]
## [1,]    1 -0.8409    1.4632 -1.9954    0.9461 -0.1137 -0.2582
## [2,]    1  1.3844    0.1877  1.1353    0.7518 -0.2951 -0.2669
## [3,]    1 -1.2555    1.0220  0.6758 -0.5174  0.9892  0.1642
```

```

## [4,] 1 0.0701 -0.5918 0.2085 0.8083 -0.7751 -0.3935
## [5,] 1 1.7114 -0.1122 -0.0578 -0.6145 0.2759 -1.8437
## [6,] 1 -0.6029 -0.9250 0.8938 1.2383 0.4108 -1.5423
## [7,] 1 -0.4722 0.7533 -0.2289 -0.3381 0.6112 -0.5862
## [8,] 1 -0.6354 -0.1126 -1.9657 1.1964 0.9366 -0.8521
## [9,] 1 -0.2858 -0.0641 -0.7535 -0.4433 -0.3675 0.7783
## [10,] 1 0.1381 0.2333 1.2802 0.1861 0.7404 -0.0303
## [11,] 1 1.2276 -1.1366 -0.9529 -2.6213 1.2185 -1.4557
## [12,] 1 -0.8018 0.8548 1.6224 2.2463 0.6291 0.0938
## [13,] 1 -1.0804 -0.5784 2.6001 0.0934 0.5277 0.9823
## [14,] 1 -0.1575 0.4964 0.1396 1.6273 -0.4723 -0.5967
## [15,] 1 -1.0718 -0.7601 -1.3507 -0.5109 0.8237 0.0748
## [16,] 1 -0.1390 -0.3414 0.7989 -0.6594 -0.4278 2.1974
## [17,] 1 -0.5973 -2.1023 -1.5550 -0.0402 -0.1426 0.7950
## [18,] 1 -2.1840 -0.3017 0.4637 -0.1187 1.4188 -0.5389
## [19,] 1 0.2408 -1.2724 0.0524 -0.0197 0.4871 -1.6013
## [20,] 1 -0.2594 -0.2797 -0.2020 -0.4857 0.6034 -0.7314
## [21,] 1 0.9005 -0.2041 1.1709 -1.4401 0.2108 -0.3557
## [22,] 1 0.9419 -0.2256 0.8848 0.1438 -0.0333 -0.9854
## [23,] 1 1.4680 0.3470 -1.3179 -1.2346 2.0252 -0.7312
## [24,] 1 0.7068 0.0324 -1.6433 -1.7525 -0.3708 1.4653
## [25,] 1 0.8190 0.4135 1.0593 -0.0355 -1.5782 1.8586
## [26,] 1 -0.2935 -0.1553 0.2901 0.3320 -0.1216 0.0035
## [27,] 1 1.4186 0.9735 -0.4000 1.5723 -1.7967 -1.3438
## [28,] 1 1.4988 0.1211 1.2431 -1.0695 -0.4756 0.1513
## [29,] 1 -0.6571 0.1892 -1.3664 0.9163 -0.8841 0.2900
## [30,] 1 -0.8528 -0.5629 -1.4414 -0.5950 -3.4981 -0.1225
## [31,] 1 0.3159 0.4984 1.3485 2.1816 -0.3820 0.1252
## [32,] 1 1.1097 -1.7423 -1.9785 -0.6838 0.9777 -0.7724
## [33,] 1 2.2155 0.9755 -1.2410 0.7501 -0.5580 -1.0130
## [34,] 1 1.2171 -0.0241 -0.1040 0.9744 -0.6265 0.9669
## [35,] 1 1.4792 0.6757 0.7330 -1.2645 -0.5305 -0.4233
## [36,] 1 0.9516 -0.7103 0.4557 -0.2774 1.8976 -0.8316
## [37,] 1 -1.0095 2.3872 0.2881 -0.1894 1.3955 1.4000
## [38,] 1 -2.0005 -0.4734 -1.0737 -0.3840 -0.7460 0.0170
## [39,] 1 -1.7622 -0.0758 0.6487 0.7406 -0.3056 1.8475
## [40,] 1 -0.1426 -0.5218 0.2992 -1.1683 1.1697 -0.6863
## [41,] 1 1.5501 0.9260 -0.7960 0.6675 0.3044 -0.2187
## [42,] 1 -0.8024 -1.0624 -0.0294 0.3662 -0.1175 0.6826
## [43,] 1 -0.0746 0.5570 2.1802 -0.5149 -0.0601 0.5241
## [44,] 1 1.8957 0.9007 0.9574 0.4506 1.4709 0.0805
## [45,] 1 -0.4566 0.9899 -0.3050 -0.1877 -1.4781 0.0538
## [46,] 1 0.5622 0.3836 -0.4184 1.3391 -0.6836 -0.7373
## [47,] 1 -0.8870 -0.3466 0.1000 0.8162 0.4605 0.9664
## [48,] 1 -0.4602 -0.5402 -0.2298 0.0822 -0.1815 0.9843
## [49,] 1 -0.7243 -0.1826 -1.4152 -0.6509 -1.1588 0.1873
## [50,] 1 -0.0692 -0.0593 -0.3926 0.7264 0.4090 0.2730

```

You should check that the function is working as intended by producing a small example that you can inspect.

We will set our ground truth $\vec{\beta}$ as an evenly spaced sequence from 0.5 to -0.5 (with 0 for the true intercept coefficient). Please review the following function:

```

generateBeta <- function(p) {
  beta_no_intercept <- pracma::linspace(0.5, -0.5, p)
  beta_1d <- c(0, beta_no_intercept)
  beta_vec <- matrix(beta_1d, ncol=1)
  return(beta_vec)
}

## example:
generateBeta(8)

```

```

##           [,1]
## [1,]  0.0000
## [2,]  0.5000
## [3,]  0.3571
## [4,]  0.2143
## [5,]  0.0714
## [6,] -0.0714
## [7,] -0.2143
## [8,] -0.3571
## [9,] -0.5000

```

Q3: complete the function `generateY()`, which generates the output vector according to our simulation data generating process. Note that `%*%` is the matrix multiplication operator in R.

```

generateY <- function(X, beta, sigma = 1) {
  ### YOUR SOLUTION HERE ###
  err <- rnorm(nrow(X), sd=sigma)
  Y <- X %*% beta + err
  return(Y)
}

X <- generateX(50,6)
b <- generateBeta(6)
Y <- generateY(X,b)

```

Next, we will introduce a mechanism to include random noise covariates into the data.

Q4: Complete the function `includeRandomCovars()` that augments `n_random_covars` to our data matrix `X`.

```

includeRandomCovars <- function(X, n_random_covars = 5) {
  ### YOUR CODE HERE ###
  covars <- matrix(rnorm(nrow(X) * n_random_covars), nrow = nrow(X))
  X_w_covars <- cbind(X, covars)

  return(X_w_covars)
}

```

```

X = generateX(50,6)
X_covariates = includeRandomCovars(X)
X_covariates

```

```

##      [,1]      [,2]      [,3]      [,4]      [,5]      [,6]      [,7]      [,8]      [,9]
## [1,]      1 -0.6110 -0.4411  0.02436 -0.2376  0.15519  0.7723  0.1007 -1.45505
## [2,]      1 -0.4635  0.5633 -0.93741 -0.9143  1.24954 -0.9945  0.8543  1.24456
## [3,]      1 -1.4817  1.1801  0.00988  1.4434  0.49689  0.3403 -0.3814 -0.43195
## [4,]      1 -0.3041 -0.9338 -1.21894 -0.0137 -0.30908 -0.4366 -0.5752  0.00687
## [5,]      1  0.3133 -0.0220  0.69063 -0.5304 -0.54923  0.1951 -0.9160  0.12455
## [6,]      1 -0.2650 -0.2332  0.46101  0.8098  0.35755 -0.0812 -0.7027 -0.40963
## [7,]      1 -1.3603 -1.2241  0.56586 -0.3352  1.08339  1.4043  0.3313  0.56341
## [8,]      1  0.3944  0.5643 -0.04236 -1.4391  0.84114  0.6614  1.5254  1.60696
## [9,]      1  0.5387  0.7181  0.49449  1.8613 -0.26448  0.5299 -0.7291 -1.00195
## [10,]     1  2.1373 -0.7720 -0.77318  0.0532 -0.84475  1.6620  2.2358  0.50933
## [11,]     1  0.2073  0.4342  1.58086  0.0484 -0.42371 -0.3625  0.4739  0.35372
## [12,]     1 -1.2766  0.2962 -1.82516 -1.2963  0.26760 -1.4411  1.6140  0.19686
## [13,]     1 -0.5157 -0.9716  0.32688 -1.0720  0.31379  1.6386  0.2252  1.02409
## [14,]     1 -0.0789 -0.6409 -0.28069 -0.0205  0.96842 -1.9234 -0.7723 -0.95398
## [15,]     1  0.6383  0.9229  0.89810 -0.7085 -0.52084  0.3098  0.2038  0.46930
## [16,]     1  0.8129 -1.0863 -0.64153  1.0239 -0.00574  0.8288  1.0974 -0.08416
## [17,]     1 -0.8932  0.4601  0.14120  1.0824  0.95289 -1.4620 -0.2684 -1.13044
## [18,]     1 -0.4016 -0.2181 -1.65451  0.5548  0.71579 -0.1035 -1.4534 -0.23049
## [19,]     1  0.8241  1.3230 -1.98473  1.5868 -0.52148  1.1319  0.2381 -0.03326
## [20,]     1 -1.3040  0.2853 -0.51433  0.2589 -0.34725 -0.6056  0.6431 -1.00304
## [21,]     1  0.8733 -0.8945 -1.96328  1.2652 -1.19362 -0.4540 -0.7084  0.07154
## [22,]     1  0.6897 -0.0178  0.30317  1.1552  0.51297 -0.0976 -0.6130  1.58742
## [23,]     1 -0.8534  0.4223  0.61376  1.6734  0.04014 -0.3371  0.1383  0.42775
## [24,]     1  0.6542  0.1234  0.74868  0.6297  0.69473  0.2502  0.6713  0.30825
## [25,]     1  1.4000 -1.6585 -0.02574 -1.1744  0.84025  0.7251  0.5605  0.12166
## [26,]     1  1.0024 -0.1279  0.71695  0.7989 -0.65504 -1.6839  1.3009 -1.31702
## [27,]     1 -3.0421 -0.8906  1.07912  1.2100  0.76273 -0.7573 -1.9296 -0.90965
## [28,]     1 -0.2423  0.8970 -0.12360 -0.5449  1.19747  0.0242 -2.2807 -0.54785
## [29,]     1  1.8518  1.3509  0.72964  1.1828 -2.17801 -1.9311 -1.2366 -0.78616
## [30,]     1 -1.6768 -1.0321  0.20995 -0.5562 -0.14911  0.3315  0.2044  0.36186
## [31,]     1 -0.6280  0.5137  0.98731  0.7100  0.24504 -0.3760  0.8921 -0.31937
## [32,]     1  0.2533  0.3397 -1.51644  2.0744 -0.21100  0.7359  0.5466  0.45333
## [33,]     1 -0.0252 -0.2641 -0.01836 -0.0827  0.30658 -0.8333 -2.1255  0.18737
## [34,]     1 -0.1866  0.4451 -0.58938 -1.1914  0.21677 -0.4801  0.2663  0.81347
## [35,]     1 -0.2334  1.3680  0.20626 -2.6048 -0.66672 -2.4066 -0.1165 -1.14920
## [36,]     1  2.7242 -0.8215  0.45214  0.4179 -0.96375  0.4129 -0.5443 -1.00233
## [37,]     1 -2.7926  0.1913  0.66605  0.1320 -1.29251 -1.1744 -0.3017  1.27034
## [38,]     1  1.6514  1.4468 -0.52698  0.4980  0.69006  0.1407 -0.0422  0.22527
## [39,]     1  2.6095 -1.0034  0.02513  0.2171  2.19732 -0.9623  0.6860 -1.42888
## [40,]     1 -0.6032  0.6154  0.55200  0.0537 -0.09927 -0.5042 -1.1069 -0.63658
## [41,]     1  0.4104  0.2446  0.41500 -0.0636 -0.57303  0.5013  2.1745  0.02427
## [42,]     1  0.5519 -0.7054  0.35403 -0.7323 -0.87683  1.0300 -0.3123  1.63999
## [43,]     1 -2.1221 -1.5612 -0.76460  1.5001  1.10753 -0.7782  0.3934  2.53712
## [44,]     1 -0.1139 -0.7577 -0.50910 -0.3028 -0.91339  0.6054 -0.3944  0.03089
## [45,]     1 -0.2737  1.5421 -1.79744  0.8536  1.22561 -1.1992  1.8395  0.16290
## [46,]     1  0.2084 -0.0647 -0.29997 -0.4017 -1.15322 -1.5588 -2.0294 -0.69681
## [47,]     1  2.3649 -0.9666 -0.45022 -0.7136 -0.80209  0.3253 -1.5579  1.86248
## [48,]     1  1.6056  0.3759  0.92636 -2.1314 -1.30125  0.5429  1.5156  0.39478

```

```

## [49,]      1  1.6539 -2.0208 -1.40134  0.5453 -0.54419  0.3044 -1.0101 -0.83124
## [50,]      1  2.2143  0.2448  1.26608  0.2174  0.86250 -0.6838  1.0897 -1.57672
##      [,10]  [,11]  [,12]
## [1,]  0.51032 -1.1882  1.1101
## [2,] -1.44637 -0.1130 -0.7689
## [3,] -1.14937 -0.5682  0.5108
## [4,] -1.45220 -2.6759  2.0151
## [5,] -0.41650 -0.1931  2.0377
## [6,]  1.72926  0.6878  1.0368
## [7,] -0.53687  1.6041  0.1132
## [8,]  1.07253  1.2065  0.3980
## [9,] -0.42842  0.7408  0.0437
## [10,]  1.00812  0.2412 -0.7922
## [11,] -1.05141  1.0528 -0.5306
## [12,]  0.41647  1.7923  1.5014
## [13,]  0.79545  1.3812  0.5265
## [14,]  1.46835  0.6548  0.2649
## [15,]  0.15662  1.8957  1.9073
## [16,] -0.00606  0.2604  0.6966
## [17,]  0.51790 -1.1271 -0.3080
## [18,] -0.81206 -1.1054 -0.4680
## [19,]  0.51814 -0.6208  0.1888
## [20,] -0.53765 -1.5298  0.6334
## [21,]  0.70986  1.0890 -0.9794
## [22,]  2.13918 -0.1529 -2.4494
## [23,]  0.70598  0.2619  0.0822
## [24,]  2.40284  0.8162  1.0446
## [25,] -0.45917 -2.2659 -0.0205
## [26,] -0.27661 -1.0405 -0.0466
## [27,] -0.91196 -0.9422 -0.2356
## [28,] -0.25316  1.1129  1.0514
## [29,] -1.34495  0.6231 -0.0282
## [30,] -0.75765  2.0115  0.6524
## [31,] -0.05960 -1.8523  1.2469
## [32,]  0.09257  0.1789  0.9464
## [33,] -0.40940 -0.3353  0.0325
## [34,]  0.06400  0.1749  0.7771
## [35,] -0.81036 -0.3020  0.6196
## [36,] -0.08616  0.4009 -0.4225
## [37,]  0.26396 -0.2015  0.4596
## [38,] -1.30783 -0.6504  0.0418
## [39,]  1.08619 -0.0315  0.3945
## [40,]  1.02517  1.2534  0.1892
## [41,] -0.60818  0.9035  0.5027
## [42,]  0.08729 -0.6828 -0.1549
## [43,]  1.47583 -1.5421  0.0641
## [44,]  2.34134  0.5705 -0.3824
## [45,] -2.00802 -1.1879  0.2209
## [46,] -0.62321 -0.3856  0.6187
## [47,]  0.45494 -1.3251 -0.1245
## [48,] -0.39179 -0.3007 -1.3126
## [49,]  0.01164 -0.2565  0.5398
## [50,] -0.22324 -0.9327 -0.4966

```

Now we have our data generating process building blocks. Please review the following function that puts everything together:

```
part1Simulation <- function(n, p, n_random_covars = 5) {
  ## main DGP:
  X <- generateX(n, p)
  beta <- generateBeta(p)
  Y <- generateY(X, beta)

  ## append random noise:
  X_w_random <- includeRandomCovars(X, n_random_covars)

  return(list(X=X, X_w_random=X_w_random, beta=beta, Y=Y))
}
```

Q5: Please complete lines marked with `TODO` in the following function that will simulate data, run a linear model on the noisy data matrix, and return R^2 and R^2_{adj} .

```
modelNoisyData <- function(n, p, n_random_covars = 5, seed = 5) {
  ## we will set seed to make sure the results are reproducible
  set.seed(seed)

  sim1_data <- part1Simulation(n, p, n_random_covars = n_random_covars)

  ## runs model on the "true" data matrix
  true_model_res <- lm(sim1_data$Y ~ ., data = sim1_data$X %>% data.frame())
  true_r2 <- summary(true_model_res)$r.squared

  ## TODO: run model on the "noisy" data matrix
  noisy_model_res <- lm(sim1_data$Y ~ ., data = sim1_data$X_w_random %>% data.frame())
  noisy_r2 <- summary(noisy_model_res)$r.squared

  ## TODO: extract adjusted R.squared from the noisy model
  noisy_adj_r2 <- summary(noisy_model_res)$adj.r.squared

  return(data.frame(
    num_random_covars=n_random_covars,
    true_r2=true_r2,
    noisy_r2=noisy_r2,
    noisy_adj_r2=noisy_adj_r2))
}
```

Great, now we have everything in place to run the full simulation and explore the affects of the noisy covariates. Let's continue to increase the number of random covariates in the linear model and look at affect on R-squared:

```
N <- 1000
P <- 20
```

```

results_data_frame <- data.frame(
  num_random_covars = numeric(),
  true_r2=numeric(),
  noisy_r2=numeric(),
  noisy_adj_r2=numeric())

for(r in seq(0, 250, 5)) {
  results_data_frame <- results_data_frame %>%
    rbind(modelNoisyData(n=N, p=P, n_random_covars=r))
}

head(results_data_frame)

```

```

##   num_random_covars true_r2 noisy_r2 noisy_adj_r2
## 1                0  0.655   0.655      0.648
## 2                 5  0.655   0.657      0.648
## 3                10  0.655   0.660      0.649
## 4                15  0.655   0.660      0.648
## 5                20  0.655   0.664      0.650
## 6                25  0.655   0.670      0.654

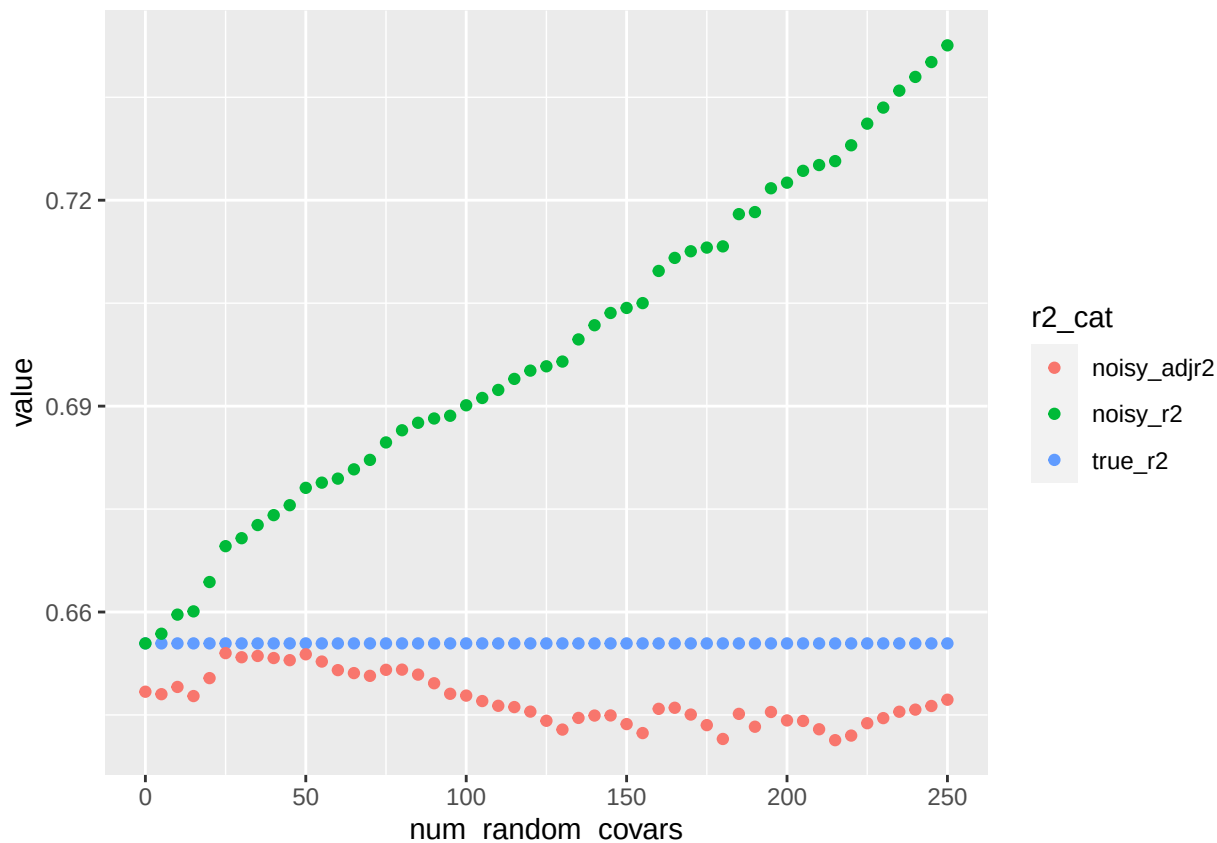
```

Q6: Reshape the data to be long in R2 categories, then plot a scatterplot of R2 values against number of random covariates, colored by the R2 category. Interpret the results.

```

### YOUR SOLUTION HERE ###
ggplot(data = gather(results_data_frame, r2_cat, value, true_r2, noisy_r2, noisy_adj_r2),
  aes(x = num_random_covars, y = value, color = r2_cat)) + geom_point()

```

YOU ANSWER HERE

Noisy R^2 monotonically increases w/ more covariates. Noisy Adj R^2 oscillates around the true adjusted R^2 .

While adjusted R-squared is valuable as a “conservative” estimate of R-squared, it might not be useful for selecting the best model. For example, in this demonstration, adjusted R-squared would have us select the model with between 25-50 completely random covariates. In a subsequent lab, we may revisit this simulation data to see if **holdout prediction performance** helps us find a better model.

Part II: Multicollinearity

Let us briefly leverage our simulation functions to explore the impacts of multicollinearity. Recall that multicollinearity is when multiple explanatory variables are highly linearly associated (ie there is substantial correlation between several predictors). We will see how including predictors correlated with β_1 will deteriorate our estimate of β_1 .

To create correlated variables, one approach is to simulate from the multivariate gaussian distribution. To circumvent this complication, we will leverage the `faux()` package, where we can specify the correlations directly vignette.

For example, here is creation of 5 covariates with $r = 0.6$ mutual correlation:

```
dat <- faux::rnorm_multi(n = 100, vars = 5, r = 0.6)
```

```
head(dat)
```

```
##      X1      X2      X3      X4      X5
## 1 -1.128  0.0291 -0.0161 -0.4267 -0.894
## 2  0.174 -0.4264 -0.2090 -0.8575 -1.191
## 3  1.088  2.0626  0.9172  0.0166 -0.697
## 4  0.432  0.8020  1.8387  0.5249  0.147
## 5 -0.467  0.1443 -1.1619 -0.2135 -1.002
## 6 -0.591 -0.2245 -0.6059  0.1995  0.407
```

```
cor(dat)
```

```
##      X1      X2      X3      X4      X5
## X1 1.000 0.641 0.686 0.587 0.618
## X2 0.641 1.000 0.655 0.646 0.562
## X3 0.686 0.655 1.000 0.591 0.637
## X4 0.587 0.646 0.591 1.000 0.609
## X5 0.618 0.562 0.637 0.609 1.000
```

Q7: Please complete lines marked with `TODO` in the following function that will simulate the data generating process, and strap-on features that are correlated with first (non-intercept) predictor, β_1 .

```
part2Simulation <- function(n, p, n_correlated_covars = 1, corr = 0.6) {
  ## TODO: generate n_correlated_covars+1 correlated variates
  correlated_variates <- faux::rnorm_multi(n = n, vars = n_correlated_covars + 1, r = corr)

  ## main DGP:
  X <- generateX(n, p)

  ## replace first non-intercept covariate with first correlated variate:
  X[,2] <- correlated_variates[,1]

  beta <- generateBeta(p)
  Y <- generateY(X, beta)

  ## TODO: bind the other correlated variates to the end of the data matrix
  X_w_correlated_variates <- cbind(X, correlated_variates[,2:ncol(correlated_variates)])

  return(list(X=X, X_w_correlated_variates=X_w_correlated_variates, beta=beta, Y=Y))
}
```

That is enough functional coding for today. Please take a minute to review the following function, that will run models, similar to part 1, and extract the estimates for β_1 under the “true” model and the colinear model:

```

modelCorrData <- function(n, p, n_correlated_covars = 1, corr = 0.6, seed = 5) {
  ## we will set seed to make sure the results are reproducible
  set.seed(seed)

  sim2_data <- part2Simulation(n, p, n_correlated_covars = n_correlated_covars, corr = corr)

  ## get true beta1
  true_beta1 <- sim2_data$beta[2]

  ## runs model on the "true" data matrix
  true_model_res <- lm(sim2_data$Y ~ ., data = sim2_data$X %>% data.frame())

  ## get estimated beta1 from the "true" model
  truemodel_beta1 <- summary(true_model_res)$coefficients[2, 1]

  ## runs correlated model
  corr_model_res <- lm(sim2_data$Y ~ ., data = sim2_data$X_w_correlated_variates %>% data.frame())

  ## get estimated beta1 from the colinear model
  colinearmodel_beta1 <- summary(corr_model_res)$coefficients[2, 1]

  return(data.frame(
    n_correlated_covars=n_correlated_covars,
    corr=corr,
    true_beta1=true_beta1,
    truemodel_beta1=truemodel_beta1,
    colinearmodel_beta1=colinearmodel_beta1))
}

### sample output:
modelCorrData(N, P, n_correlated_covars = 2, corr = 0.6)

```

```

##   n_correlated_covars corr true_beta1 truemodel_beta1 colinearmodel_beta1
## 1                    2 0.6         0.5         0.571         0.576

```

And the following will run our function many times, so that we can report the standard deviation of β_{a1} estimates. Do not worry if you cannot parse through this function.

```

corrDataBetaVar <- function(n, p, n_correlated_covars = 1, corr = 0.6, runs = 250) {
  results_list <-
    lapply(seq_len(runs), function(i) {
      modelCorrData(n, p, n_correlated_covars = n_correlated_covars, corr = corr, seed = i)})

  sds_df <-
    Reduce(rbind, results_list) %>% dplyr::group_by(n_correlated_covars, corr, true_beta1) %>%
    dplyr::summarise_all(sd)

  names(sds_df)[4:5] <- c("truemodel_beta1_sd", "colinearmodel_beta1_sd")

  return(sds_df)
}

```

```
corrDataBetaVar(N, P, n_correlated_covars = 2, corr = 0.9)
```

```
## # A tibble: 1 x 5
## # Groups:   n_correlated_covars, corr [1]
##   n_correlated_covars  corr true_beta1 truemodel_beta1_sd colinearmodel_beta1_sd
##               <dbl> <dbl>      <dbl>          <dbl>          <dbl>
## 1                   2  0.9         0.5            0.0328          0.0791
```

Similar to Part 1, we will use our functions to examine the affect of magnitude of correlation and number of correlated covariates below. These may take a little while to run.

```
N <- 250
P <- 20

results_data_frame_cor <- data.frame(
  n_correlated_covars = numeric(),
  corr=numeric(),
  true_beta1=numeric(),
  truemodel_beta1_sd=numeric(),
  colinearmodel_beta1_sd=numeric())

results_data_frame_ncovar <- results_data_frame_cor

### Dynamics by correlation magnitude:
for(r in seq(0.5, 0.9, 0.05)) {
  results_data_frame_cor <- results_data_frame_cor %>%
    rbind(corrDataBetaVar(N, P, n_correlated_covars = 1, corr = r))
}

### Dynamics by number covariates:
for(c in seq(1, 30, 5)) {
  results_data_frame_ncovar <- results_data_frame_ncovar %>%
    rbind(corrDataBetaVar(N, P, n_correlated_covars = c, corr = 0.7))
}
```

```
results_data_frame_cor
```

```
## # A tibble: 9 x 5
## # Groups:   n_correlated_covars, corr [9]
##   n_correlated_covars  corr true_beta1 truemodel_beta1_sd colinearmodel_beta1_sd
##               <dbl> <dbl>      <dbl>          <dbl>          <dbl>
## 1                   1  0.5         0.5            0.0606          0.0699
## 2                   1 0.55         0.5            0.0607          0.0726
## 3                   1  0.6         0.5            0.0608          0.0760
## 4                   1 0.65         0.5            0.0610          0.0802
## 5                   1  0.7         0.5            0.0612          0.0856
## 6                   1 0.75         0.5            0.0614          0.0928
## 7                   1  0.8         0.5            0.0616          0.103
## 8                   1 0.85         0.5            0.0619          0.118
## 9                   1  0.9         0.5            0.0623          0.143
```

```
results_data_frame_ncovar
```

```
## # A tibble: 6 x 5
## # Groups:   n_correlated_covars, corr [6]
##   n_correlated_covars  corr true_beta1 truemodel_beta1_sd colinearmodel_beta1_sd
##           <dbl> <dbl>      <dbl>          <dbl>          <dbl>
## 1             1  0.7         0.5          0.0612          0.0856
## 2             6  0.7         0.5          0.0637          0.105
## 3            11  0.7         0.5          0.0685          0.115
## 4            16  0.7         0.5          0.0676          0.122
## 5            21  0.7         0.5          0.0692          0.124
## 6            26  0.7         0.5          0.0679          0.122
```

In essence, we are re-running the simulation many times, and reporting the standard deviation of $\hat{\beta}_1$ for the “true” data matrix and the “colinear” data matrix, manipulating either the magnitude of the correlation or the number of extra correlated variates.

Q8: In your own words, summarize what the results in objects: `results_data_frame_cor` and `results_data_frame_ncovar` are telling us about multicollinearity.

When changing the number of correlated covariance and keeping the correlation between them constant, the $\hat{\beta}_1$ standard deviation initially increases and then plateaus/decreases. When we change the correlation but keep the number of colinear covariates constant, the standard deviation deviation monotonically increases.

One last thing to note is that the colinear variables have no causal impact on Y . We can exacerbate the issue of colinearity by coding the colinear variables to have nonzero β values.

You finished!